

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационная безопасность

Лабораторная работа №1

Разработка защищенного REST API с интеграцией в CI/CD

Выполнил:

Черемисова Мария

Группа: Р3410

Санкт-Петербург,

2026

ЦЕЛЬ И УСТРОЙСТВО ПРОЕКТА

Цель работы — создать REST API с JWT-аутентификацией, доступом к собственным записям, защитой от SQL-инъекций и XSS и автоматическими проверками кода и зависимостей.

Проект написан на Java 17 и Spring Boot 4.1.1. Maven Wrapper 3.9.9 выполняет сборку по pom.xml. Код находится в src/main/java, интеграционные тесты — в src/test/java, CI — в .github/workflows/ci.yml. Пакеты auth, user, post, config и common разделяют задачи приложения.

Данные хранятся через Spring Data JPA во встроенной H2 в памяти. После остановки приложения таблицы удаляются, поэтому регистрацию нужно повторить. Локальный Git-репозиторий создан; origin и публичный репозиторий пока не настроены.

МЕТОДЫ REST API

Приложение предоставляет четыре HTTP-метода. Тела запросов и ответов передаются в JSON; защищённые методы принимают заголовок Authorization: Bearer <JWT>.

Метод	Адрес	Доступ	Успешный результат
POST	/auth/register	Открытый	201 Created, без тела
POST	/auth/login	Открытый	200 OK, JWT в JSON
GET	/api/data	Bearer JWT	200 OK, до 100 своих записей
POST	/api/data	Bearer JWT	201 Created, созданная запись

Регистрация принимает username (3–32 латинских букв, цифр, точек, дефисов или подчёркиваний) и password (12–72 символа, не более 72 байт UTF-8). Ответ - 201 без токена; JWT выдаётся при входе.

```
{"username":"student1","password":"long-unique-password-1"}
```

POST /auth/login принимает те же поля. При верной паре логин/пароль возвращается accessToken, тип Bearer и срок действия 900 секунд. В примере ниже <JWT> обозначает место для фактического токена.

```
{"accessToken":"<JWT>","tokenType":"Bearer","expiresInSeconds":900}
```

POST /api/data принимает title и content. В ответе есть id, экранированные title и content, createdAt. Владелец определяется только по проверенному JWT и не передаётся в запросе. GET /api/data выбирает последние 100 записей текущего пользователя по убыванию id.

{"title":"Моя запись","content":"Текст для лабораторной"}

РЕАЛИЗОВАННЫЕ МЕРЫ ЗАЩИТЫ

Защита от SQL-инъекций

Методы UserRepository и PostRepository формируют запросы через Spring Data JPA. Логин и владелец передаются как параметры; SQL не собирается конкатенацией строк из пользовательского ввода. Сами записи также сохраняются через JPA. Поэтому текст клиента не может изменить структуру выполняемого SQL-запроса.

Защита от XSS

API возвращает JSON, а не HTML-страницы. При формировании PostResponse сервис DataService применяет HtmlUtils.htmlEscape к title и content. Например, введённое `<script>` возвращается как `<script>`. Это снижает риск при последующем отображении текста в HTML; если появится отдельный веб-интерфейс, в нём также потребуется безопасный вывод с учётом контекста.

Пароли, JWT и доступ к данным

Пароль при регистрации кодируется BCryptPasswordEncoder с фактором стоимости 12. В H2 сохраняется только хэш, а при входе пароль сравнивается с ним через PasswordEncoder. Для неизвестного пользователя сервис тоже выполняет проверку BCrypt с фиктивным хэшем и возвращает общую ошибку «Неверный логин или пароль».

После успешного входа JwtService создаёт JWT с алгоритмом HS256 на 15 минут. В токене есть subject пользователя, issuer secure-api-lab, время выдачи и окончания срока. Ключ приходит из переменной JWT_SECRET_BASE64; приложение требует не менее 32 байт после Base64-декодирования. Секретный ключ не хранится в pom.xml или application.properties.

Spring Security работает без серверной HTTP-сессии. Только /auth/register и /auth/login открыты без токена; остальные адреса требуют аутентификации. OAuth2 Resource Server проверяет Bearer JWT, подпись, срок действия и issuer. DataController получает подтверждённый subject и передаёт его в DataService; сервис выбирает и создаёт записи именно этого пользователя. Запрос без токена или с неверным токеном получает 401 Unauthorized.

Валидация и ответы об ошибках

Bean Validation ограничивает пустые и слишком длинные поля. ApiExceptionHandler возвращает короткие сообщения и подходящие HTTP-статусы: 400 при некорректном JSON или полях, 401 при неверных учётных данных, 409 при повторном логине. Настройки приложения не включают stack trace в стандартные ошибки.

ПОЛУЧЕННЫЕ ОТЧЕТЫ ПО SAST/SCA ИЗ ACTIONS

SAST:

```
[INFO] The original artifact has been renamed to /home/runner/work/lb_lr1/lb_lr1/target/secure-api-lab-1.0.0.jar.original
[INFO]
[INFO] >>> spotbugs:4.10.4.1:check (default-cli) > :spotbugs @ secure-api-lab >>>
[INFO]
[INFO] --- spotbugs:4.10.4.1:spotbugs (spotbugs) @ secure-api-lab ---
[INFO] Done SpotBugs Analysis...
[INFO] Skipping com.github.spotbugs:spotbugs-maven-plugin:4.10.4.1:spotbugs report goal
[INFO]
[INFO] <<< spotbugs:4.10.4.1:check (default-cli) < :spotbugs @ secure-api-lab <<<
[INFO]
[INFO]
[INFO] --- spotbugs:4.10.4.1:check (default-cli) @ secure-api-lab ---
[INFO] BugInstance size is 0
[INFO] Error size is 0
[INFO] No errors/warnings found
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 24.002 s
[INFO] Finished at: 2026-09-30T10:38:55Z
[INFO] -----
```

SCA:

```
Scan dependencies 1h 43m 59s
204 dependency-check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may
    exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition,
    and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is
    at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection
    with the use of this tool, the analysis performed, or the resulting report.
265
266
267 About ODC: https://dependency-check.github.io/DependencyCheck/general/internals.html
268 False Positives: https://dependency-check.github.io/DependencyCheck/general/suppression.html
269
270
271 [INFO] Analysis Started
272 [INFO] Finished Archive Analyzer (0 seconds)
273 [INFO] Finished File Name Analyzer (0 seconds)
274 [INFO] Finished Jar Analyzer (0 seconds)
275 [INFO] Finished Dependency Merging Analyzer (0 seconds)
276 [INFO] Finished Hint Analyzer (0 seconds)
277 [INFO] Finished Version Filter Analyzer (0 seconds)
278 [INFO] Created CPE Index (1 seconds)
279 [INFO] Finished CPE Analyzer (3 seconds)
280 [INFO] Finished False Positive Analyzer (0 seconds)
281 [INFO] Finished NVD CVE Analyzer (0 seconds)
282 [INFO] Finished RetireJS Analyzer (0 seconds)
283 [WARNING] Sonatype OSS Index Analyzer disabled due to missing credentials. Authentication with token is now required, and OSS Index is
    migrating to Sonatype Guide. See https://dependency-check.github.io/DependencyCheck/analyzers/oss-index-analyzer.html for more information on
    authentication with Sonatype Guide OSS Index.
284 [INFO] Finished Vulnerability Suppression Analyzer (0 seconds)
285 [INFO] Finished Known Exploited Vulnerability Analyzer (0 seconds)
286 [INFO] Finished Dependency Bundling Analyzer (0 seconds)
287 [INFO] Finished Unused Suppression Rule Analyzer (0 seconds)
288 [INFO] Analysis Complete (5 seconds)
289 [INFO] Writing HTML report to: /home/runner/work/lb_lr1/lb_lr1/target/dependency-check-report.html
290 [INFO] -----
291 [INFO] BUILD SUCCESS
292 [INFO] -----
293 [INFO] Total time: 01:43 h
294 [INFO] Finished at: 2026-09-30T12:22:30Z
295 [INFO] -----
```

Вывод

Разработан REST API на Java и Spring Boot с регистрацией, JWT-аутентификацией и доступом только к собственным записям. SQL-запросы выполняются через Spring Data JPA, пользовательский текст экранируется перед возвратом, пароли хранятся в виде BCrypt-хэшей. Локально прошли интеграционные тесты и SpotBugs. Для полного оформления работы остаётся опубликовать репозиторий, настроить NVD_API_KEY и получить реальные результаты SCA и GitHub Actions.